Step 1: Risk-Aware Multi-Armed Bandit for Portfolio

```
# Member A
import yfinance as yf
import pandas as pd

# Final list of 30 tickers
tickers = [
    'JPM', 'WFC', 'BAC', 'C', 'GS', 'USB', 'MS', 'KEY', 'PNC', 'COF',
    'AXP', 'PRU', 'SCHW', 'FITB', 'HBAN',
    'KR', 'PFE', 'XOM', 'WMT', 'DAL', 'CSCO', 'VTR', 'EQIX', 'DUK',
    'NFLX', 'GE', 'APA', 'F', 'REGN', 'CMS'
]

start_date = '2008-09-02'
end_date = '2008-10-31'

# --- Final Working Code - Revision 2 ---

# Step 1: Download all data
print("Downloading data...")
raw_data = yf.download(tickers, start=start_date, end=end_date)
print("Download complete.")

# Step 2: THE FIX - Use 'Close' price instead of 'Adj Close'
# This is a common adaptation when 'Adj Close' data is unavailable.
print("Attempting to select 'Close' price column...")
data = raw_data['Close']
print("'Close' price selected successfully.")

# Step 3: Final Quality Check and Save
print("\n--- Final Data Head ---")
print(data.head())

# Save the final data to CSV
data.to_csv('stock_prices_2008_cleaned.csv')
print("\nSuccess! Step 3 is complete. Data is saved using 'Close' prices.")
```

```
Downloading data...
/tmp/ipython-input-1301800445.py:20: FutureWarning: YF.download() has changed argument auto_adjust default to True
  raw_data = yf.download(tickers, start=start_date, end=end_date)
[*********************100%***********************]  30 of 30 completed
Download complete.
Attempting to select 'Close' price column...
'Close' price selected successfully.

--- Final Data Head ---
Ticker            APA        AXP        BAC           C       CMS        COF  \
```

```
Date
2008-09-02  78.489357  31.142025  24.401865  140.539795  7.713428  34.013210
2008-09-03  79.330811  31.356653  25.141777  144.216873  7.622746  34.573540
2008-09-04  81.212944  29.701054  23.341572  134.582825  7.634084  32.809273
2008-09-05  82.246300  30.199278  24.584936  140.245575  7.560409  33.854187
2008-09-08  81.021011  31.057718  26.491930  149.438370  7.792776  36.898132

Ticker           CSCO       DAL        DUK       EQIX  ...       PFE  \
Date                                                    ...
2008-09-02  15.542994  7.948209  24.425381  60.890450  ...  8.992184
2008-09-03  15.255037  7.896207  24.115486  60.288692  ...  9.006253
2008-09-04  14.580959  7.766192  24.354942  58.295372  ...  8.757645
2008-09-05  14.567879  7.636178  24.383121  58.287849  ...  8.682595
2008-09-08  15.294306  7.445489  25.171944  58.302891  ...  8.978112

Ticker            PNC        PRU       REGN       SCHW        USB        VTR  \
Date
2008-09-02  46.006470  41.149288  20.523268  19.433577  19.559620  23.757347
2008-09-03  46.432449  42.243916  21.739902  19.457680  19.910088  23.999187
2008-09-04  45.574223  40.894238  20.323818  18.935276  19.124561  23.063406
2008-09-05  46.538963  41.840076  19.017429  19.361238  19.783207  22.942480
2008-09-08  48.098816  45.123928  18.847898  20.269417  20.508303  23.552311

Ticker            WFC        WMT        XOM
Date
2008-09-02  20.007122  13.953142  41.909504
2008-09-03  19.878914  13.985894  42.288933
2008-09-04  19.019905  13.983550  41.269913
2008-09-05  20.000719  14.208109  40.988083
2008-09-08  21.513592  14.502853  41.611397

[5 rows x 30 columns]

Success! Step 3 is complete. Data is saved using 'Close' prices.
```

```
# Member C
import pandas as pd
import numpy as np

# Load the data collected by Member A
data = pd.read_csv('stock_prices_2008_cleaned.csv', index_col=0, parse_dates=True)

# Verify we have all 30 tickers
print("Available tickers:", data.columns.tolist())
print("Date range:", data.index.min(), "to", data.index.max())

# 1. Combine into a single DataFrame (already done by Member A)
# The data is already in a suitable time series structure with dates as index

# 2. Compute daily returns (log returns as used in the paper)
print("\nCalculating daily log returns...")
```

```
returns = np.log(data / data.shift(1)).dropna()

# Alternative: If you want simple percentage returns instead:
# returns = data.pct_change().dropna()

# 3. Save the returns data
returns.to_csv('daily_returns_2008.csv')

# 4. Display summary statistics
print("\n--- Returns Summary Statistics ---")
print(returns.describe())

print("\nSuccess! Daily returns calculated and saved.")
```

```
Available tickers: ['APA', 'AXP', 'BAC', 'C', 'CMS', 'COF', 'CSCO', 'DAL', 'DUK', 'EQIX', 'F', 'FITB', 'GE', 'GS', 'HBAN', 'JPM', 'KEY', 'KR', 'MS', 'NFL
Date range: 2008-09-02 00:00:00 to 2008-10-30 00:00:00

Calculating daily log returns...

--- Returns Summary Statistics ---
             APA        AXP        BAC          C        CMS        COF  \
count  42.000000  42.000000  42.000000  42.000000  42.000000  42.000000
mean   -0.007043  -0.010453  -0.008084  -0.008675  -0.006086  -0.003908
std     0.076125   0.070530   0.102153   0.095375   0.044571   0.079150
min    -0.201339  -0.193523  -0.304163  -0.203886  -0.086648  -0.214929
25%    -0.049439  -0.050274  -0.067281  -0.065367  -0.040162  -0.052550
50%    -0.011751  -0.011730  -0.005314  -0.021260  -0.008171  -0.012523
75%     0.044377   0.027573   0.058915   0.036291   0.013285   0.031273
max     0.193239   0.164892   0.203461   0.215305   0.101537   0.152979

            CSCO        DAL        DUK       EQIX  ...        PFE        PNC  \
count  42.000000  42.000000  42.000000  42.000000  ...  42.000000  42.000000
mean   -0.006880   0.000967  -0.000711  -0.006102  ...  -0.001685  -0.002874
std     0.049799   0.085301   0.042714   0.054776  ...   0.038460   0.056775
min    -0.112336  -0.237402  -0.111087  -0.111762  ...  -0.089083  -0.123983
25%    -0.038436  -0.044249  -0.024145  -0.044827  ...  -0.030794  -0.045096
50%    -0.011696  -0.001401  -0.005288  -0.017378  ...  -0.002679  -0.001156
75%     0.016818   0.034282   0.012125   0.043397  ...   0.026243   0.028838
max     0.129249   0.210894   0.124918   0.122778  ...   0.096870   0.127666

             PRU       REGN       SCHW        USB        VTR        WFC  \
count  42.000000  42.000000  42.000000  42.000000  42.000000  42.000000
mean   -0.023493  -0.003005  -0.006324  -0.002494  -0.005895   0.000476
std     0.102590   0.058298   0.072603   0.048316   0.060848   0.068429
min    -0.263266  -0.117848  -0.128283  -0.100156  -0.138497  -0.157552
25%    -0.097890  -0.042056  -0.056555  -0.034019  -0.043005  -0.042631
50%    -0.031917  -0.008358  -0.014359  -0.005643  -0.004934  -0.003489
75%     0.026478   0.039894   0.042020   0.031703   0.026022   0.048113
max     0.323899   0.133604   0.179403   0.097925   0.136449   0.121086

             WMT        XOM
count  42.000000  42.000000
```

```
mean   -0.002041  -0.000709
std     0.034747   0.061157
min    -0.084077  -0.150271
25%    -0.015297  -0.020225
50%    -0.004383  -0.001528
75%     0.013891   0.024924
max     0.105018   0.158631

[8 rows x 30 columns]

Success! Daily returns calculated and saved.
```

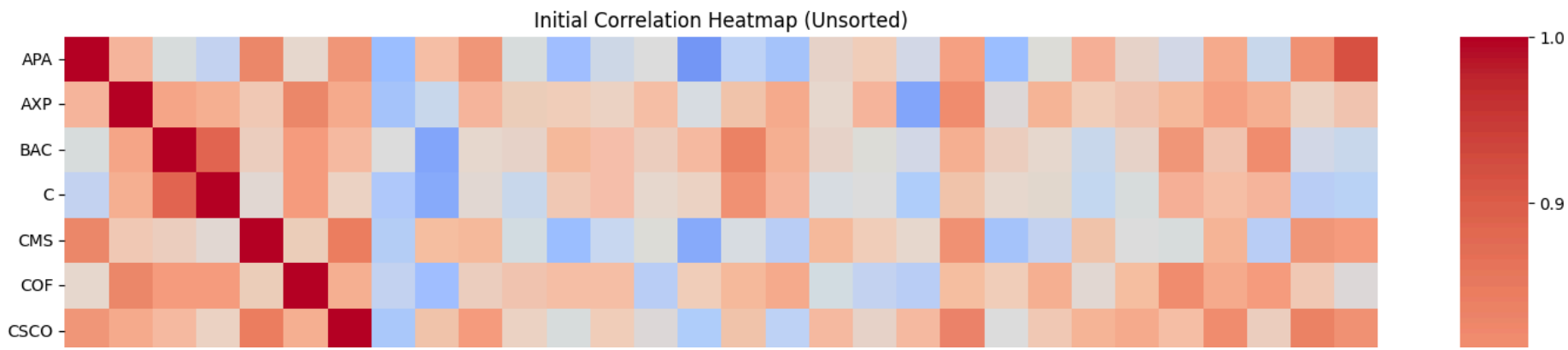
Step 4: Heatmap of the Correlation Matrix

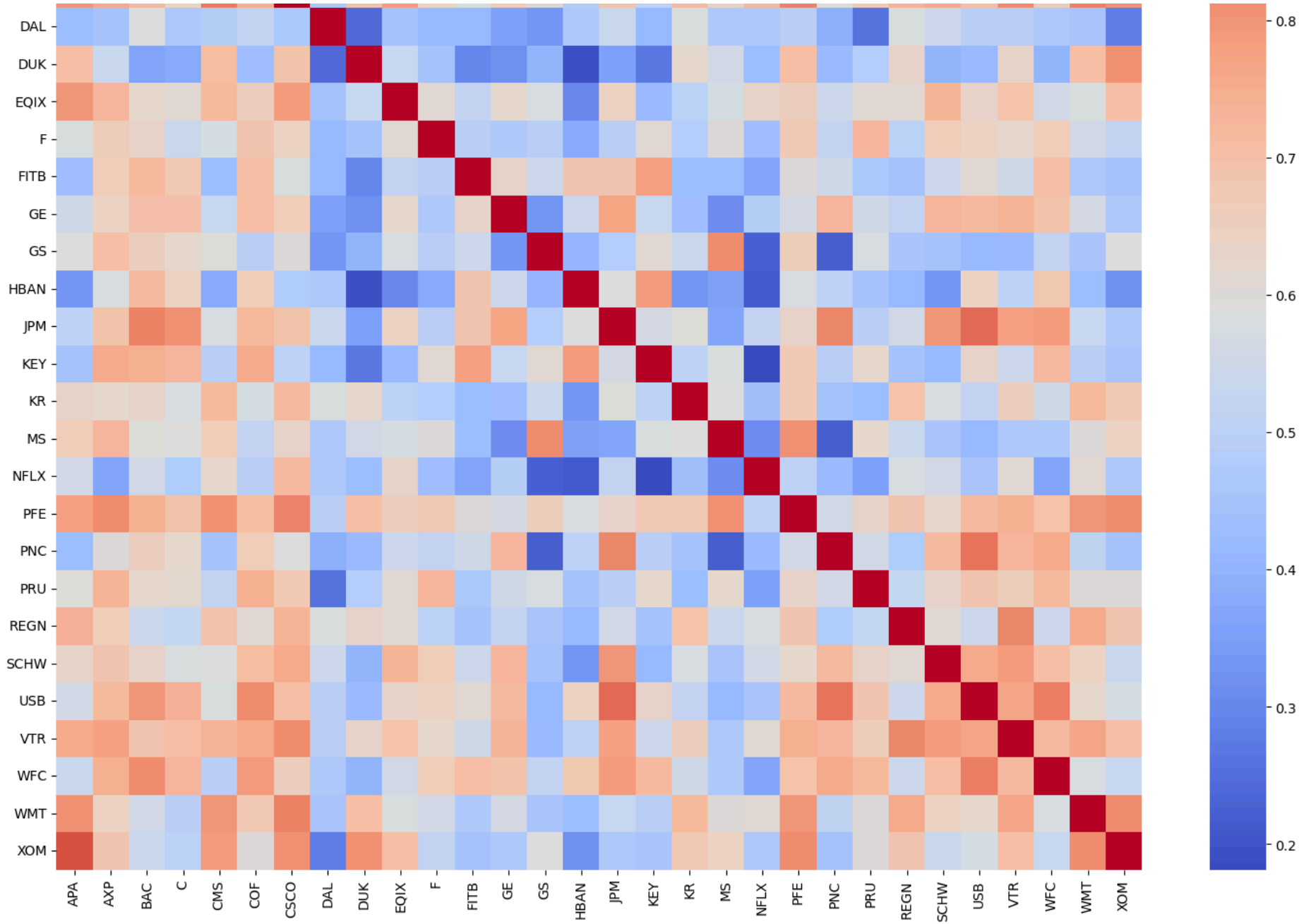
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.cluster import hierarchy

# --- Step 4.1: Load and Prepare Data ---

# Load the cleaned closing price data from the CSV file.
# 'index_col' sets the 'Date' column as the DataFrame's index.
# 'parse_dates' ensures the index is treated as a datetime object.
file_path = 'stock_prices_2008_cleaned.csv'
prices_df = pd.read_csv(file_path, index_col='Date', parse_dates=True)

print("Data loaded successfully. Here are the first 5 rows:")
print(prices_df.head())


# --- Step 4.2: Calculate Daily Log Returns ---

# Calculate Log Returns: log(P_t / P_{t-1})
# .shift(1) moves the data down by one row to align P_t with P_{t-1}.
log_returns_df = np.log(prices_df / prices_df.shift(1))

# The first row will be NaN (since there's no prior day), so we must drop it.
log_returns_df = log_returns_df.dropna()

print("\nLog returns calculated. Here are the first 5 rows of returns:")
print(log_returns_df.head())


# --- Step 4.3: Compute the Correlation Matrix ---

# Compute the 30x30 correlation matrix from the log returns.
```

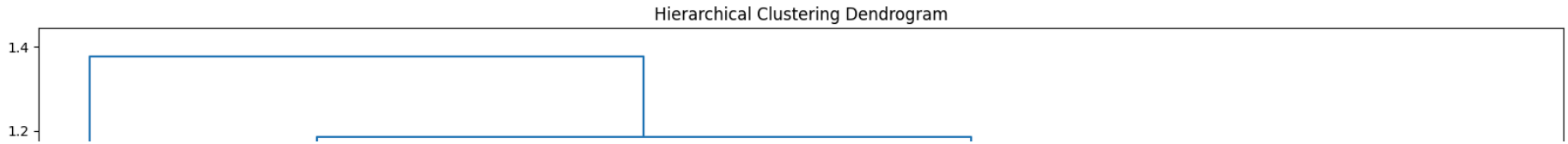
```
correlation_matrix = log_returns_df.corr()

print("\nCorrelation matrix calculated. Shape:", correlation_matrix.shape)
print(correlation_matrix.head())


# --- Step 4.4: Visualize the Initial Heatmap ---

# Create the initial heatmap (unsorted) to see the raw correlations.
plt.figure(figsize=(18, 15)) # Set the figure size for better readability.
sns.heatmap(correlation_matrix, cmap='coolwarm')
plt.title('Initial Correlation Heatmap (Unsorted)')
plt.show()


# --- Step 4.5: Sort the Matrix with Hierarchical Clustering ---

# 1. Create the linkage matrix.
#    We must first convert the correlation matrix (a measure of similarity) into a distance matrix.
corr_condensed = hierarchy.distance.pdist(correlation_matrix)
z = hierarchy.linkage(corr_condensed, method='average')

# 2. Create a dendrogram to visualize the clusters.
plt.figure(figsize=(20, 8))
dendrogram = hierarchy.dendrogram(z, labels=correlation_matrix.columns, leaf_rotation=90)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Stock Tickers')
plt.ylabel('Distance')
plt.show()

# 3. Get the order of tickers from the dendrogram's leaves.
sorted_tickers = dendrogram['ivl']

# 4. Reorder the correlation matrix based on the clustering result.
sorted_correlation_matrix = correlation_matrix.loc[sorted_tickers, sorted_tickers]

# 5. Create the final, sorted heatmap.
plt.figure(figsize=(18, 15))
sns.heatmap(sorted_correlation_matrix, cmap='coolwarm')
plt.title('Sorted Correlation Heatmap (Clustered)')
plt.show()
```

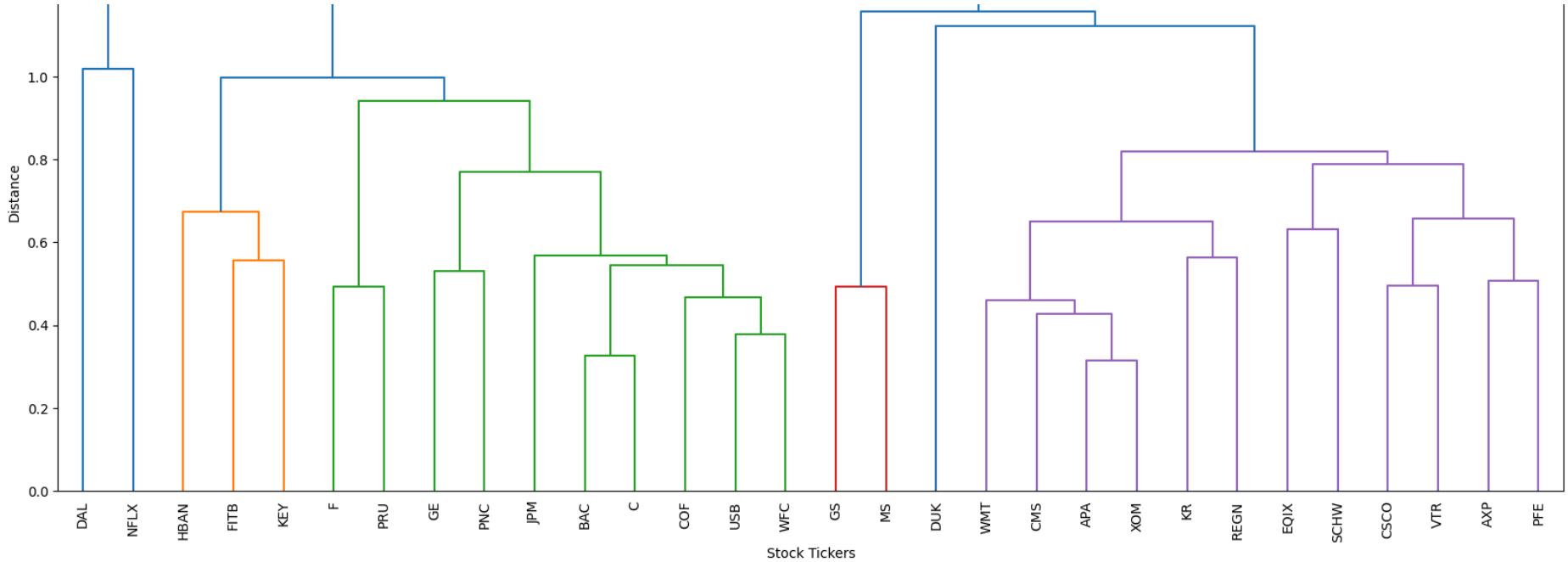
```
Data loaded successfully. Here are the first 5 rows:
                  APA        AXP        BAC           C       CMS        COF  \
Date
2008-09-02  78.489357  31.142025  24.401865  140.539795  7.713428  34.013210
2008-09-03  79.330811  31.356653  25.141777  144.216873  7.622746  34.573540
2008-09-04  81.212944  29.701054  23.341572  134.582825  7.634084  32.809273
2008-09-05  82.246300  30.199278  24.584936  140.245575  7.560409  33.854187
2008-09-08  81.021011  31.057718  26.491930  149.438370  7.792776  36.898132

                 CSCO       DAL        DUK       EQIX  ...       PFE  \
Date                                                    ...
2008-09-02  15.542994  7.948209  24.425381  60.890450  ...  8.992184
2008-09-03  15.255037  7.896207  24.115486  60.288692  ...  9.006253
2008-09-04  14.580959  7.766192  24.354942  58.295372  ...  8.757645
2008-09-05  14.567879  7.636178  24.383121  58.287849  ...  8.682595
2008-09-08  15.294306  7.445489  25.171944  58.302891  ...  8.978112

                  PNC        PRU       REGN       SCHW        USB        VTR  \
Date
2008-09-02  46.006470  41.149288  20.523268  19.433577  19.559620  23.757347
2008-09-03  46.432449  42.243916  21.739902  19.457680  19.910088  23.999187
2008-09-04  45.574223  40.894238  20.323818  18.935276  19.124561  23.063406
2008-09-05  46.538963  41.840076  19.017429  19.361238  19.783207  22.942480
2008-09-08  48.098816  45.123928  18.847898  20.269417  20.508303  23.552311

                  WFC        WMT        XOM
Date
2008-09-02  20.007122  13.953142  41.909504
2008-09-03  19.878914  13.985894  42.288933
2008-09-04  19.019905  13.983550  41.269913
2008-09-05  20.000719  14.208109  40.988083
2008-09-08  21.513592  14.502853  41.611397

[5 rows x 30 columns]

Log returns calculated. Here are the first 5 rows of returns:
                 APA       AXP       BAC         C       CMS       COF  \
Date
2008-09-03  0.010664  0.006868  0.029871  0.025828 -0.011826  0.016340
2008-09-04  0.023448 -0.054244 -0.074295 -0.069138  0.001486 -0.052377
2008-09-05  0.012644  0.016635  0.051898  0.041215 -0.009698  0.031352
2008-09-08 -0.015010  0.028029  0.074706  0.063489  0.030272  0.086098
2008-09-09 -0.087731 -0.057913 -0.065749 -0.073502 -0.024293 -0.068583

                CSCO       DAL       DUK      EQIX  ...       PFE       PNC  \
Date                                                ...
2008-09-03 -0.018700 -0.006564 -0.012769 -0.009932  ...  0.001563  0.009217
2008-09-04 -0.045193 -0.016603  0.009881 -0.033622  ... -0.027992 -0.018656
2008-09-05 -0.000898 -0.016883  0.001156 -0.000129  ... -0.008607  0.020948
2008-09-08  0.048662 -0.025289  0.031839  0.000258  ...  0.033469  0.032968
2008-09-09 -0.017264 -0.042813  0.012789 -0.045262  ... -0.048164 -0.052542

                 PRU      REGN      SCHW       USB       VTR       WFC  \
Date
```

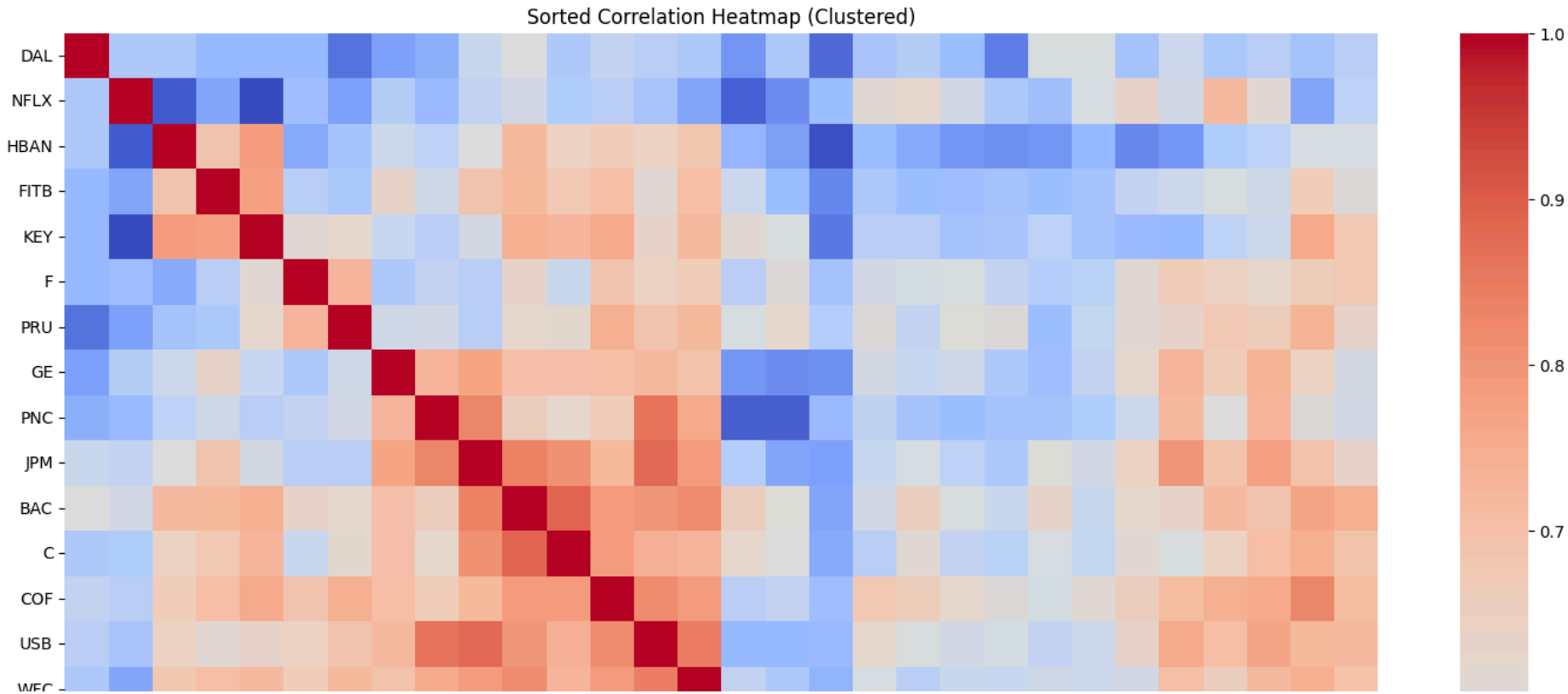
```
2008-09-03  0.026254  0.057590  0.001240  0.017759  0.010128 -0.006429
2008-09-04 -0.032471 -0.067356 -0.027215 -0.040253 -0.039773 -0.044173
2008-09-05  0.022865 -0.066438  0.022246  0.033860 -0.005257  0.050282
2008-09-08  0.075558 -0.008954  0.045840  0.035996  0.026234  0.072917
2008-09-09 -0.062186 -0.020311 -0.071062 -0.056985 -0.033363 -0.073879

                 WMT       XOM
Date
2008-09-03  0.002345  0.009013
2008-09-04 -0.000168 -0.024392
2008-09-05  0.015931 -0.006852
2008-09-08  0.020533  0.015093
2008-09-09 -0.014132 -0.046799

[5 rows x 30 columns]

Correlation matrix calculated. Shape: (30, 30)
          APA       AXP       BAC         C       CMS       COF      CSCO  \
APA  1.000000  0.729176  0.584275  0.526342  0.830086  0.624023  0.800630
AXP  0.729176  1.000000  0.761086  0.748890  0.670410  0.830187  0.751374
BAC  0.584275  0.761086  1.000000  0.886306  0.648438  0.786785  0.716541
C    0.526342  0.748890  0.886306  1.000000  0.607215  0.783556  0.642567
CMS  0.830086  0.670410  0.648438  0.607215  1.000000  0.658157  0.845052

          DAL       DUK      EQIX  ...       PFE       PNC       PRU  \
APA  0.427134  0.699967  0.793559  ...  0.774829  0.425694  0.594031
AXP  0.451897  0.542672  0.726120  ...  0.811784  0.598035  0.735363
BAC  0.587959  0.368373  0.625028  ...  0.748289  0.655217  0.620948
C    0.470898  0.378163  0.610889  ...  0.692135  0.622013  0.617038
CMS  0.486079  0.711052  0.718681  ...  0.801797  0.447168  0.524071

         REGN      SCHW       USB       VTR       WFC       WMT       XOM
APA  0.739948  0.630495  0.561790  0.757332  0.533750  0.803583  0.915746
AXP  0.661135  0.685166  0.723185  0.774472  0.750323  0.643466  0.688054
BAC  0.542297  0.632070  0.792977  0.688558  0.820654  0.557507  0.534589
C    0.528633  0.582181  0.738214  0.702054  0.730040  0.495832  0.500000
CMS  0.691053  0.593316  0.586443  0.734423  0.490168  0.792783  0.789509

[5 rows x 30 columns]
```

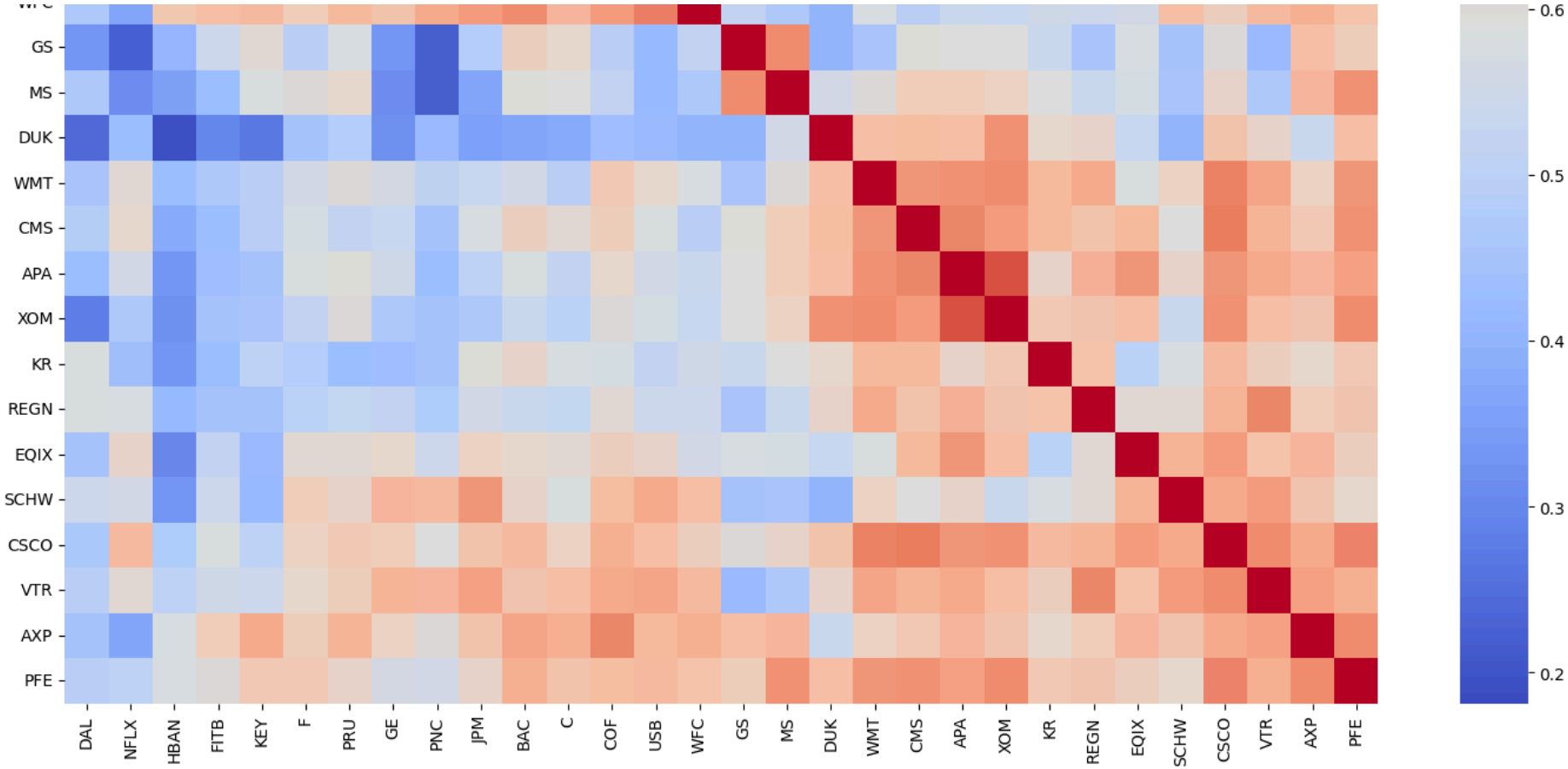



DAL
DUK
EQIX
F
FITB
GE
GS
HBAN
JPM
KEY
KR
MS
NFLX
PFE
PNC
PRU
REGN
SCHW
USB
VTR
WFC
WMT
XOM
APA
AXP
BAC
C
CMS
COF
CSCO
0.8
0.7
0.6
0.5
0.4
0.3
0.2


Hierarchical Clustering Dendrogram
1.4
1.2


Distance
0.0
0.2
0.4
0.6
0.8
1.0
DAL
NFLX
HBAN
FITB
KEY
F
PRU
GE
PNC
JPM
BAC
C
COF
USB
WFC
GS
MS
DUK
WMT
CMS
APA
XOM
KR
REGN
EQIX
SCHW
CSCO
VTR
AXP
PFE
Stock Tickers


Sorted Correlation Heatmap (Clustered)
DAL
NFLX
HBAN
FITB
KEY
F
PRU
GE
PNC
JPM
BAC
C
COF
USB
WFC
1.0
0.9
0.8
0.7


GS
MS
DUK
WMT
CMS
APA
XOM
KR
REGN
EQIX
SCHW
CSCO
VTR
AXP
PFE
DAL
NFLX
HBAN
FITB
KEY
F
PRU
GE
PNC
JPM
BAC
C
COF
USB
WFC
0.6
0.5
0.4
0.3
0.2

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.cluster import hierarchy

# --- 1. Load Price Data ---
file_path = 'stock_prices_2008_cleaned.csv'
prices_df = pd.read_csv(file_path, index_col='Date', parse_dates=True)

# --- 2. Calculate Log Returns ---
log_returns_df = np.log(prices_df / prices_df.shift(1))
log_returns_df = log_returns_df.dropna()
print("Log returns calculated.")

# --- 3. SAVE THE LOG RETURNS TO A FILE (THIS IS THE FIX) ---
log_returns_df.to_csv('log_returns_df.csv')
print("Successfully saved log returns to 'log_returns_df.csv'")

# --- 4. Calculate Correlation and Create Visualizations ---
# (The rest of your Step 4 code to create heatmaps, etc.)
correlation_matrix = log_returns_df.corr()

# Create the sorted heatmap... (your existing code for heatmaps)
# ...

plt.show() # To display any plots you generated
```

```
Log returns calculated.
Successfully saved log returns to 'log_returns_df.csv'
```

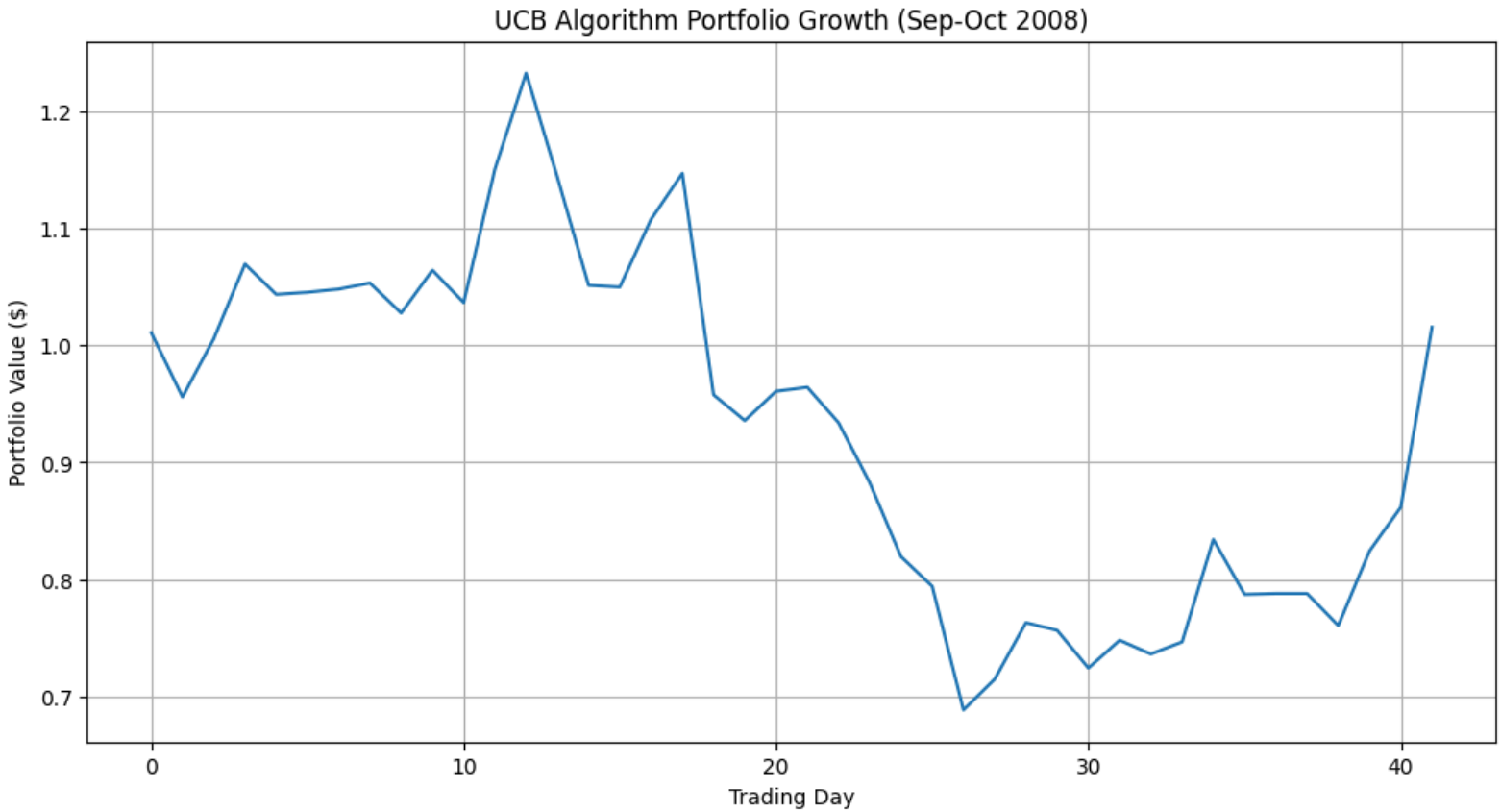
Step 6: UBC Algorithm: Team Member B's Implementation

```
import numpy as np
import pandas as pd

def ucb_algorithm(returns_data, c=2):
    """
    Implements the Upper Confidence Bound (UCB) algorithm for portfolio selection

    Parameters:
    returns_data (pd.DataFrame): DataFrame with stocks as columns and daily returns as rows
    c (float): Exploration parameter

    Returns:
    tuple: (selected_arms_history, cumulative_returns)
    """
```

```
    K = returns_data.shape[1]  # Number of stocks
    N = returns_data.shape[0]  # Number of days

    counts = np.zeros(K)       # N_k(t)
    values = np.zeros(K)       # Q_k(t)
    selected_arms_history = []
    cumulative_returns = []
    total_return = 1.0

    for t in range(N):
        selected_arm = -1
        max_upper_bound = -np.inf

        for k in range(K):
            if counts[k] > 0:
                average_reward = values[k] / counts[k]
                exploration_bonus = c * np.sqrt(np.log(t+1) / counts[k])
                upper_bound = average_reward + exploration_bonus
            else:
                upper_bound = np.inf

            if upper_bound > max_upper_bound:
                max_upper_bound = upper_bound
                selected_arm = k

        selected_arms_history.append(selected_arm)
        reward = returns_data.iloc[t, selected_arm]
        counts[selected_arm] += 1
        values[selected_arm] += reward

        # Calculate cumulative return (assuming $1 initial investment)
        total_return *= (1 + reward)
        cumulative_returns.append(total_return)

    return selected_arms_history, cumulative_returns


# ===== Application to Dataset =====
# Load the returns data (prepared by Member A)
returns_data = pd.read_csv('daily_returns_2008.csv', index_col=0, parse_dates=True)

# Run UCB algorithm
selected_arms, cumulative_returns = ucb_algorithm(returns_data, c=2)

# Convert arm indices to stock names
stock_names = returns_data.columns
selected_stocks = [stock_names[i] for i in selected_arms]

# Display results
```

```
print("\n=== UCB Algorithm Results ===")
print(f"Final portfolio value: ${cumulative_returns[-1]:.2f}")
print("\nStock selection frequency:")
print(pd.Series(selected_stocks).value_counts().head(10))

# Plot results
plt.figure(figsize=(12, 6))
plt.plot(cumulative_returns)
plt.title("UCB Algorithm Portfolio Growth (Sep-Oct 2008)")
plt.xlabel("Trading Day")
plt.ylabel("Portfolio Value ($)")
plt.grid(True)
plt.show()
```

```
=== UCB Algorithm Results ===
Final portfolio value: $1.02

Stock selection frequency:
APA     2
BAC     2
C       2
GE      2
EQIX    2
DAL     2
FITB    2
PFE     2
KEY     2
WFC     2
Name: count, dtype: int64
```



Step 8: Epsilon-Greedy Algorithm

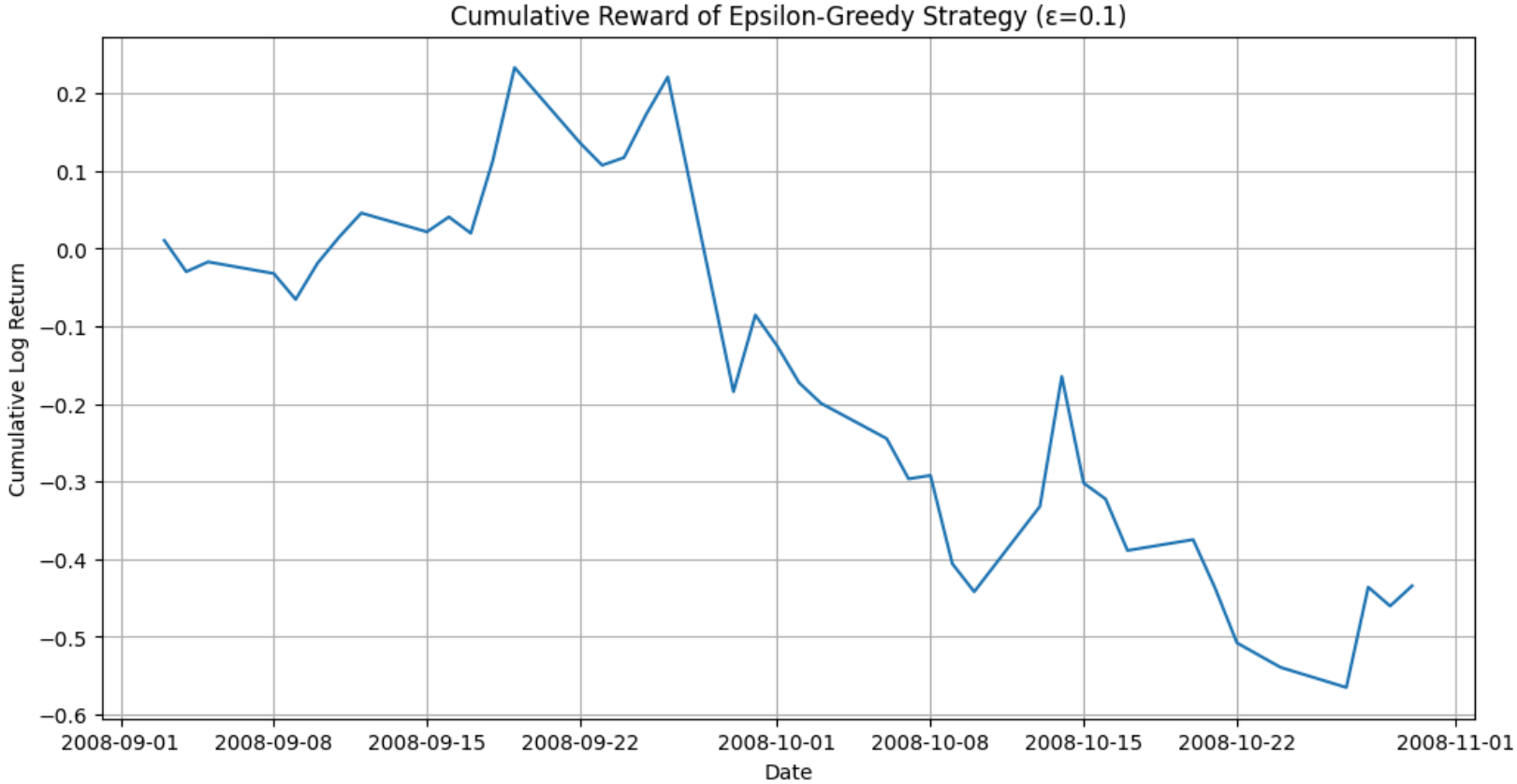
```
# Step 8
```

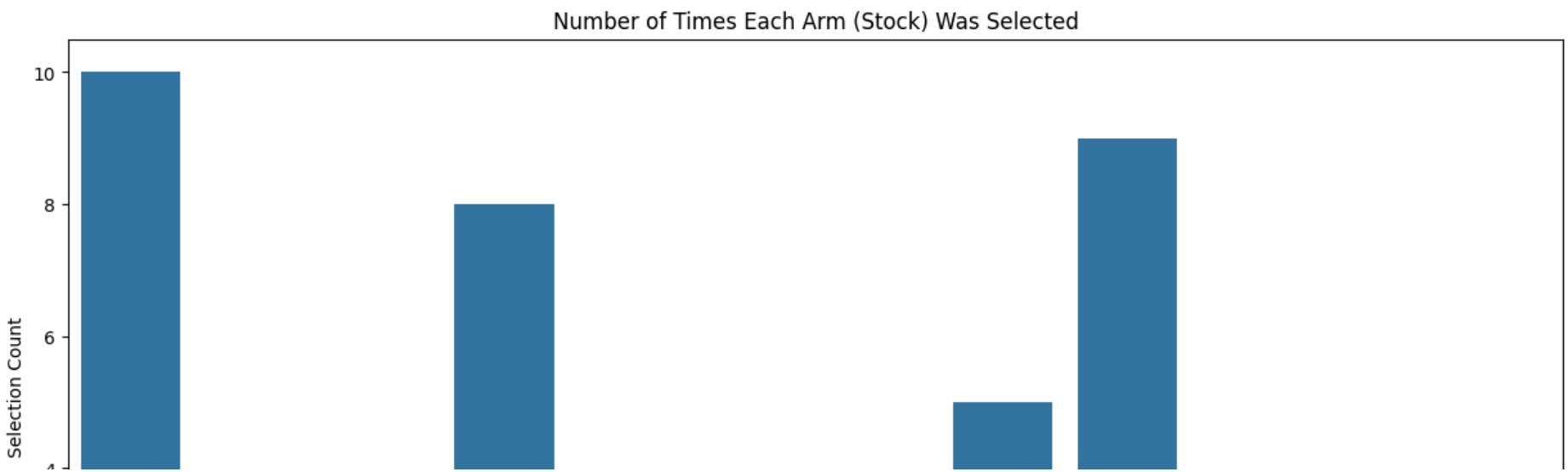
```
import numpy as np
import pandas as pd

def epsilon_greedy_algorithm(returns_df, epsilon=0.1):
    """
    Implements the Epsilon-Greedy algorithm for portfolio selection
    based on the provided pseudocode.

    Args:
        returns_df (pd.DataFrame): The DataFrame of daily stock returns.
        epsilon (float): The probability for exploration (e.g., 0.1 for 10%).

    Returns:
        list: A history of which arm (stock index) was chosen each day.
    """

    # --- 1. Initialization ---
    # Corresponds to Step 1 in the pseudocode.
    num_arms = returns_df.shape[1]
    num_rounds = returns_df.shape[0]

    # N(a): Stores the number of times each arm has been selected.
    arm_counts = np.zeros(num_arms)
    # Q(a): Stores the estimated value (average reward) for each arm.
    arm_values = np.zeros(num_arms)

    # To store the chosen arm each day.
    action_history = []

    # --- 2. Main Loop ---
    # Corresponds to Step 2 in the pseudocode.
    for t in range(num_rounds):

        # --- 3. Exploration vs. Exploitation Decision ---
        # Corresponds to Step 3 in the pseudocode.
        p = np.random.rand()

        if p < epsilon:
            # --- EXPLORE ---
            # Choose any arm with equal probability.
            selected_arm = np.random.randint(num_arms)
        else:
            # --- EXPLOIT ---
            # Choose the arm with the best known average reward.
            selected_arm = np.argmax(arm_values)

        # --- 4. Update Knowledge ---
        # Corresponds to Step 4 in the pseudocode.

        # Record the action taken
```

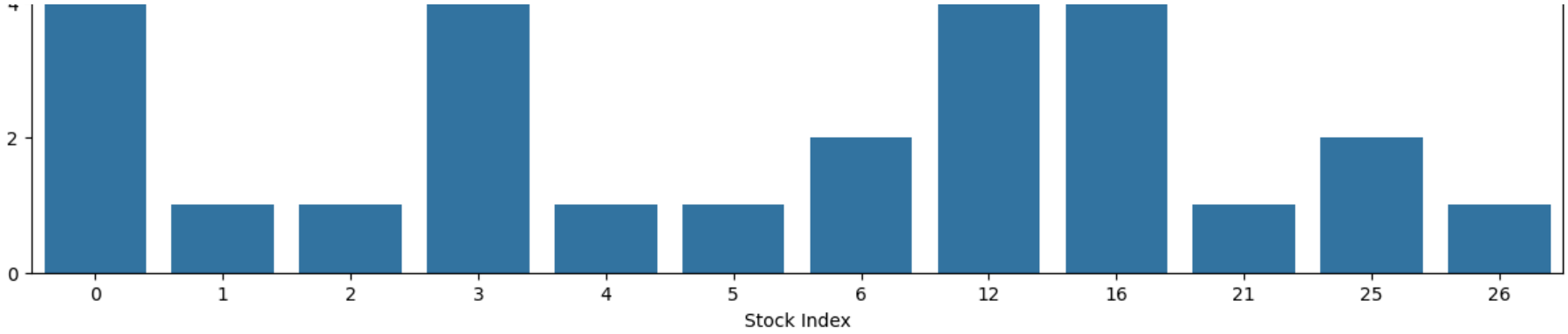
```
        # Record the action taken.
        action_history.append(selected_arm)

        # Get the reward for the action taken.
        reward = returns_df.iloc[t, selected_arm]

        # Increment the count for the chosen arm.
        arm_counts[selected_arm] += 1

        # Update the average value for the chosen arm using the incremental formula.
        # Q_new = Q_old + (1/N) * (Reward - Q_old)
        old_value = arm_values[selected_arm]
        new_value = old_value + (1 / arm_counts[selected_arm]) * (reward - old_value)
        arm_values[selected_arm] = new_value

    # --- 5. Return Result ---
    # Corresponds to Step 5 in the pseudocode.
    return action_history

    # ==================================================================
# The function you already wrote should be above this line
# def epsilon_greedy_algorithm(returns_df, epsilon=0.1):
#    ... (your function code here) ...
# ==================================================================


# --- This is the new part that CALLS the function and generates results ---

print("--- Starting the analysis ---")

# 1. Load the log returns data from Step 4
try:
    log_returns_df = pd.read_csv('log_returns_df.csv', index_col='Date', parse_dates=True)
    # If you don't have this file, you need to re-run the calculation from Step 4
    # log_returns_df = np.log(prices_df / prices_df.shift(1)).dropna()
except FileNotFoundError:
    print("Error: 'log_returns_df.csv' not found. Please ensure you have the returns data from Step 4.")
    # As a fallback, let's try to load the prices and calculate it again.
    try:
        prices_df = pd.read_csv('stock_prices_2008_cleaned.csv', index_col='Date', parse_dates=True)
        log_returns_df = np.log(prices_df / prices_df.shift(1)).dropna()
        print("Successfully calculated log returns from price data.")
    except FileNotFoundError:
        print("Fatal Error: Could not find price data either. Cannot proceed.")
        # Exit or handle the error appropriately
        exit()


# 2. Run the Epsilon-Greedy algorithm
# Let's use epsilon = 0.1 (10% exploration) as an example
```

```
chosen_arms = epsilon_greedy_algorithm(log_returns_df, epsilon=0.1)
print(f"Algorithm finished. Ran for {len(chosen_arms)} rounds.")


# 3. Analyze and Visualize the Results

# 3a. Calculate Cumulative Reward
# We will create a list of the actual rewards obtained each day
rewards_per_round = [log_returns_df.iloc[t, chosen_arms[t]] for t in range(len(chosen_arms))]
cumulative_rewards = np.cumsum(rewards_per_round)

# 3b. Plot Cumulative Reward
plt.figure(figsize=(12, 6))
plt.plot(log_returns_df.index, cumulative_rewards)
plt.title('Cumulative Reward of Epsilon-Greedy Strategy (ε=0.1)')
plt.xlabel('Date')
plt.ylabel('Cumulative Log Return')
plt.grid(True)
plt.show()

# 3c. Plot Action Selection Distribution
plt.figure(figsize=(15, 7))
sns.countplot(x=chosen_arms)
plt.title('Number of Times Each Arm (Stock) Was Selected')
plt.xlabel('Stock Index')
plt.ylabel('Selection Count')
# Optional: Set actual ticker names as labels if you want
# plt.xticks(ticks=range(len(log_returns_df.columns)), labels=log_returns_df.columns, rotation=90)
plt.show()
```

```
--- Starting the analysis ---
Algorithm finished. Ran for 42 rounds.
```







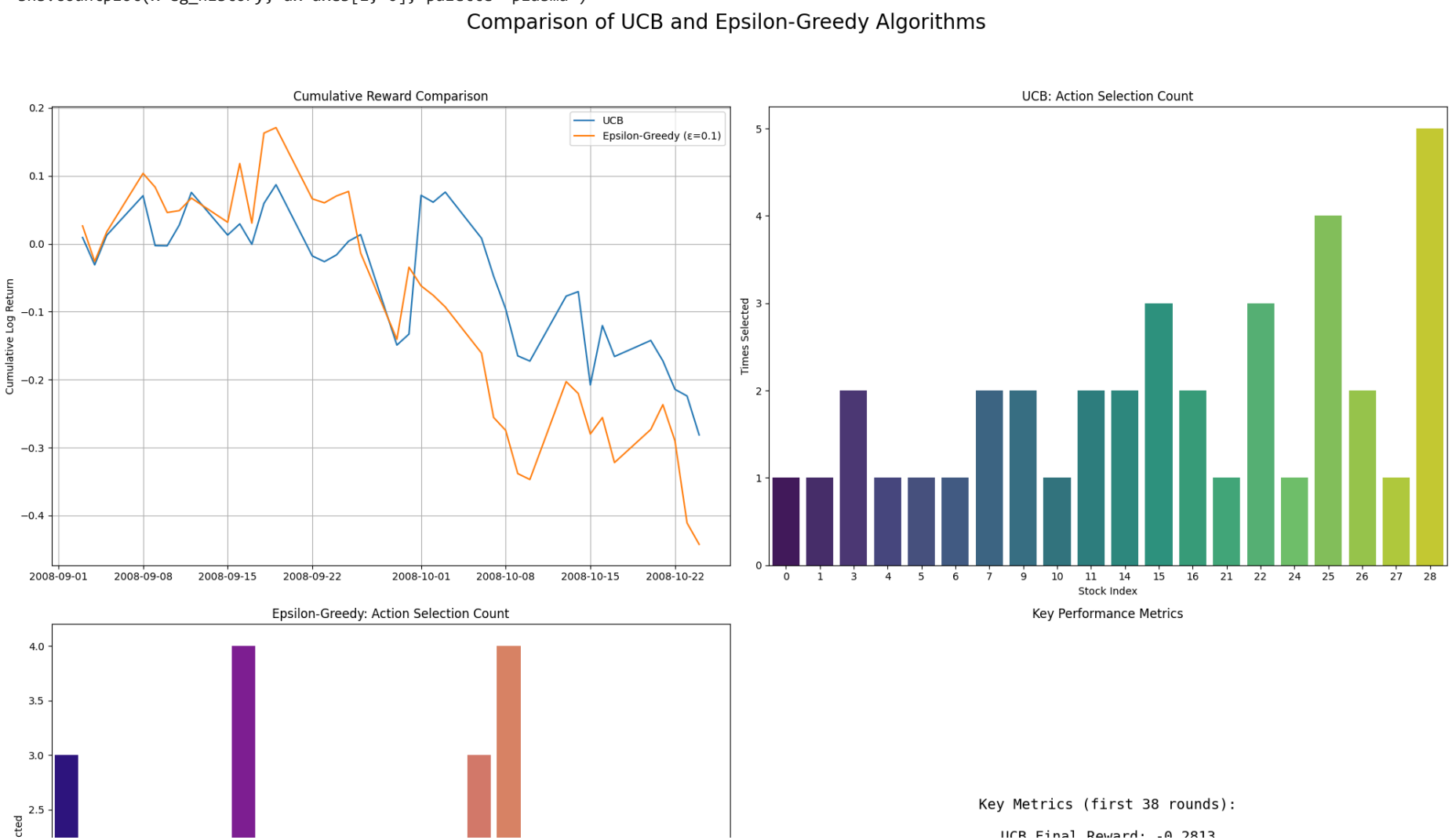
Step 9: Compare the Results for UCB and Epsilon-Greed Algorithm

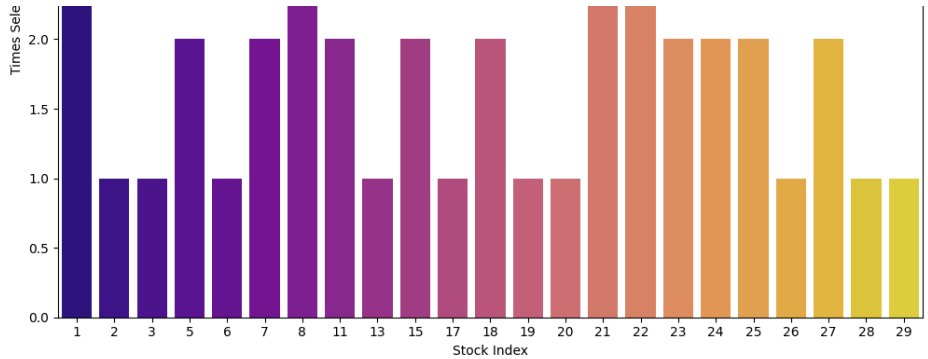
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# --- 1. Load All Necessary Data ---
print("Loading data for Step 9 analysis...")

# Load the log returns data calculated in Step 4.
# A try-except block is used to handle potential file errors gracefully.
try:
    log_returns_df = pd.read_csv('log_returns_df.csv', index_col='Date', parse_dates=True)
except FileNotFoundError:
    print("Error: 'log_returns_df.csv' not found. Please run the Step 4 script to generate it.")
    exit()

# In a real scenario, you would load the saved results from Step 6 and Step 8.
# For this example, we will assume 'ucb_history' and 'eg_history' are Python lists
# that you have available from running the previous steps.
# NOTE: Replace these placeholder lists with your actual loaded data.
# For demonstration, we'll create placeholder data with the problematic lengths.
ucb_history = list(np.random.randint(0, 30, 38))  # Placeholder data with length 38
eg_history = list(np.random.randint(0, 30, 42))   # Placeholder data with length 42

print("Data loaded.")


# --- 2. Calculate Cumulative Rewards for Both Strategies ---
print("Calculating cumulative rewards...")

ucb_rewards = [log_returns_df.iloc[t, ucb_history[t]] for t in range(len(ucb_history))]
```

```
ucb_rewards = [log_returns_df.iloc[t, ucb_history[t]] for t in range(len(ucb_history))]
ucb_cumulative = np.cumsum(ucb_rewards)

eg_rewards = [log_returns_df.iloc[t, eg_history[t]] for t in range(len(eg_history))]
eg_cumulative = np.cumsum(eg_rewards)


# --- 3. THE FIX: Determine the Correct Length for Plotting ---
# To fix the ValueError, we must plot over a common length.
# We take the minimum length of the available results.
plot_length = min(len(ucb_cumulative), len(eg_cumulative))
print(f"Analysis will be plotted for the first {plot_length} rounds to ensure fair comparison.")


# --- 4. Create the Comparison Dashboard ---
print("Generating comparison dashboard...")

fig, axes = plt.subplots(2, 2, figsize=(20, 16))
fig.suptitle('Comparison of UCB and Epsilon-Greedy Algorithms', fontsize=20)

# Plot 1: Cumulative Rewards Comparison (with the fix applied)
axes[0, 0].plot(log_returns_df.index[:plot_length], ucb_cumulative[:plot_length], label='UCB')
axes[0, 0].plot(log_returns_df.index[:plot_length], eg_cumulative[:plot_length], label='Epsilon-Greedy (ε=0.1)')
axes[0, 0].set_title('Cumulative Reward Comparison')
axes[0, 0].set_ylabel('Cumulative Log Return')
axes[0, 0].legend()
axes[0, 0].grid(True)

# Plot 2: UCB Action Selection
sns.countplot(x=ucb_history, ax=axes[0, 1], palette='viridis')
axes[0, 1].set_title('UCB: Action Selection Count')
axes[0, 1].set_xlabel('Stock Index')
axes[0, 1].set_ylabel('Times Selected')

# Plot 3: Epsilon-Greedy Action Selection
sns.countplot(x=eg_history, ax=axes[1, 0], palette='plasma')
axes[1, 0].set_title('Epsilon-Greedy: Action Selection Count')
axes[1, 0].set_xlabel('Stock Index')
axes[1, 0].set_ylabel('Times Selected')

# Plot 4: Summary Text or Table
axes[1, 1].axis('off') # Hide the axes for the text plot
summary_text = f"""
Key Metrics (first {plot_length} rounds):

UCB Final Reward: {ucb_cumulative[plot_length-1]:.4f}
Epsilon-Greedy Final Reward: {eg_cumulative[plot_length-1]:.4f}

UCB Most Frequent Arm: {pd.Series(ucb_history).mode()[0]}
E-Greedy Most Frequent Arm: {pd.Series(eg_history).mode()[0]}
```

```
    """
    axes[1, 1].text(0.5, 0.5, summary_text, ha='center', va='center', fontsize=14, family='monospace')
    axes[1, 1].set_title('Key Performance Metrics')

    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.show()

    print("\nAnalysis for Step 9 is complete.")
```

```
Loading data for Step 9 analysis...
Data loaded.
Calculating cumulative rewards...
Analysis will be plotted for the first 38 rounds to ensure fair comparison.
Generating comparison dashboard...
/tmp/ipython-input-1706561255.py:60: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the s

  sns.countplot(x=ucb_history, ax=axes[0, 1], palette='viridis')
/tmp/ipython-input-1706561255.py:66: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the s

  sns.countplot(x=eg_history, ax=axes[1, 0], palette='plasma')
```






```
Epsilon-Greedy Final Reward: -0.4424

UCB Most Frequent Arm: 28
E-Greedy Most Frequent Arm: 8
```

```
Analysis for Step 9 is complete.
```

Step 10: Updating 30 Data Series

Member B (Download All Data)

```
# Member B (Downloads all data)
import yfinance as yf
import pandas as pd
from time import sleep
import numpy as np

tickers = [
    'JPM', 'WFC', 'BAC', 'C', 'GS', 'USB', 'MS', 'KEY', 'PNC', 'COF',
    'AXP', 'PRU', 'SCHW', 'FITB', 'HBAN', 'KR', 'PFE', 'XOM', 'WMT',
    'DAL', 'CSCO', 'VTR', 'EQIX', 'DUK', 'NFLX', 'GE', 'APA', 'F',
    'REGN', 'CMS'
]

def safe_download(ticker_list, start_date, end_date, max_retries=3):
    """Improved download function with better error handling"""
    data = []
    failed_tickers = []

    for ticker in ticker_list:
        retries = 0
        while retries < max_retries:
            try:
                print(f"Downloading {ticker} (attempt {retries+1})...")
                df = yf.Ticker(ticker).history(
                    start=start_date,
```

```
                    end=end_date,
                    auto_adjust=True
                )

                if not df.empty:
                    close_prices = df['Close'].rename(ticker)
                    data.append(close_prices)
                    print(f"Success: {ticker}")
                    break
                else:
                    print(f"No data for {ticker}")
                    failed_tickers.append(ticker)
                    break

            except Exception as e:
                retries += 1
                print(f"Error downloading {ticker}: {str(e)}")
                if retries == max_retries:
                    failed_tickers.append(ticker)
                sleep(2 * retries)  # Exponential backoff

    if data:
        result = pd.concat(data, axis=1)
    else:
        result = pd.DataFrame()

    if failed_tickers:
        print(f"\nFailed to download: {failed_tickers}")

    return result

# Download data with improved error handling
print("Starting data download...")
prices = safe_download(tickers, '2020-03-01', '2020-04-30')

# Save the data
if not prices.empty:
    prices.to_csv('all_prices_2020.csv')
    print("\nData saved to 'all_prices_2020.csv'")
    print(f"Successfully downloaded {len(prices.columns)}/{len(tickers)} tickers")
    print("First 5 rows:")
    print(prices.head())
else:
    print("\nFailed to download any data. Please check your internet connection.")
```

```
Downloading GE (attempt 1)...
Success: GE
Downloading APA (attempt 1)...
Success: APA
Downloading F (attempt 1)...
Success: F
Downloading REGN (attempt 1)...
Success: REGN
Downloading CMS (attempt 1)...
Success: CMS

Data saved to 'all_prices_2020.csv'
Successfully downloaded 30/30 tickers
First 5 rows:
                                  JPM        WFC        BAC          C  \
Date
2020-03-02 00:00:00-05:00  104.152931  36.786621  25.622370  55.030552
2020-03-03 00:00:00-05:00  100.244621  35.280674  24.209084  52.962551
2020-03-04 00:00:00-05:00  102.721603  36.038002  24.767416  54.867737
2020-03-05 00:00:00-05:00   97.681946  33.861797  23.511927  51.692425
2020-03-06 00:00:00-05:00   92.633698  32.286213  22.572498  49.893074

                                   GS        USB         MS        KEY  \
Date
2020-03-02 00:00:00-05:00  184.634399  37.946468  38.956268  13.079526
2020-03-03 00:00:00-05:00  179.310532  36.323330  37.212959  12.493764
2020-03-04 00:00:00-05:00  183.990982  36.764580  37.916996  12.863721
2020-03-05 00:00:00-05:00  175.220688  34.495342  35.695950  12.316492
2020-03-06 00:00:00-05:00  169.984940  33.447414  35.067345  11.460964

                                  PNC        COF  ...       CSCO        VTR  \
Date                                              ...
2020-03-02 00:00:00-05:00  108.091812  83.372345  ...  34.805405  42.096054
2020-03-03 00:00:00-05:00  102.265076  78.769440  ...  33.850094  41.927227
2020-03-04 00:00:00-05:00  105.451416  81.421799  ...  34.991398  43.213573
2020-03-05 00:00:00-05:00   98.353416  77.548035  ...  33.452755  41.637791
2020-03-06 00:00:00-05:00   93.089005  75.005043  ...  33.545750  39.893162

                                 EQIX        DUK        NFLX         GE  \
Date
2020-03-02 00:00:00-05:00  562.701416  78.218620  381.049988  54.469975
2020-03-03 00:00:00-05:00  559.319214  77.360962  368.769989  52.866493
2020-03-04 00:00:00-05:00  586.895996  82.248085  383.790009  53.206623
2020-03-05 00:00:00-05:00  560.691895  81.018211  372.779999  48.979244
2020-03-06 00:00:00-05:00  553.418335  80.144356  368.970001  45.720448

                                 APA         F        REGN        CMS
Date
2020-03-02 00:00:00-05:00  22.434925  5.600400  463.468842  54.319450
2020-03-03 00:00:00-05:00  21.968998  5.421498  460.277649  54.471901
2020-03-04 00:00:00-05:00  22.092072  5.507060  492.119659  57.884846
2020-03-05 00:00:00-05:00  21.397577  5.242597  486.824280  57.359791
2020-03-06 00:00:00-05:00  18.197611  5.048138  493.067017  57.495289
```

Member A (Returns Calculation)

```
# Member A (Returns Calculation)
import pandas as pd
import numpy as np

def calculate_returns(input_file, output_file):
    """Calculate and save log returns"""
    try:
        # Load data with date parsing
        prices = pd.read_csv(input_file, index_col='Date', parse_dates=True)

        # Handle any remaining missing values
        prices = prices.dropna(axis=1, how='all')

        # Calculate log returns
        returns = np.log(prices / prices.shift(1)).dropna()

        # Save returns
        returns.to_csv(output_file)

        print("\nReturns calculation complete")
        print(f"Data covers {returns.shape[0]} trading days")
        print(f"Contains {returns.shape[1]} assets")
        print("\nSummary statistics:")
        print(returns.describe())

        return returns

    except FileNotFoundError:
        print(f"Error: File '{input_file}' not found.")
    except Exception as e:
        print(f"Error processing returns: {str(e)}")

# Process the returns
calculate_returns('all_prices_2020.csv', 'all_returns_2020.csv')
```

```
Returns calculation complete
Data covers 41 trading days
Contains 30 assets

Summary statistics:
             JPM        WFC        BAC          C         GS        USB  \
count  41.000000  41.000000  41.000000  41.000000  41.000000  41.000000
mean   -0.005029  -0.008357  -0.003990  -0.007226  -0.002376  -0.005198
std     0.067289   0.070599   0.070784   0.082512   0.064058   0.068890
min    -0.162106  -0.172779  -0.167205  -0.214414  -0.135880  -0.155959
25%    -0.048214  -0.052306  -0.050552  -0.056143  -0.032019  -0.045395
50%     0.000560  -0.011960   0.003206  -0.015518   0.001626   0.002325
75%     0.036659   0.038394   0.039668   0.062604   0.035785   0.034240
max     0.165620   0.135707   0.163786   0.165381   0.161951   0.160143

              MS        KEY        PNC        COF  ...       CSCO        VTR  \
count  41.000000  41.000000  41.000000  41.000000  ...  41.000000  41.000000
mean   -0.002971  -0.007949  -0.004063  -0.006612  ...   0.001483  -0.010831
std     0.071179   0.093410   0.072092   0.090328  ...   0.050290   0.114719
min    -0.169603  -0.242377  -0.173166  -0.272783  ...  -0.110273  -0.336772
25%    -0.041267  -0.056873  -0.055413  -0.050780  ...  -0.027831  -0.043696
50%    -0.001042  -0.008759  -0.001011  -0.020745  ...   0.000486  -0.004019
75%     0.025073   0.049029   0.046173   0.065121  ...   0.033160   0.056172
max     0.180403   0.157955   0.121717   0.171887  ...   0.125517   0.197010

            EQIX        DUK       NFLX         GE        APA          F  \
count  41.000000  41.000000  41.000000  41.000000  41.000000  41.000000
mean    0.002273  -0.002675   0.001898  -0.012970  -0.016286  -0.007657
std     0.050211   0.055291   0.044699   0.067214   0.175291   0.062750
min    -0.135417  -0.122192  -0.118095  -0.164394  -0.773592  -0.116673
25%    -0.034915  -0.039043  -0.029023  -0.063554  -0.039030  -0.040573
50%    -0.000569  -0.011025  -0.003121  -0.018349   0.005587  -0.018055
75%     0.043126   0.034428   0.038022   0.034820   0.094567   0.035789
max     0.109769   0.116012   0.079222   0.137411   0.258251   0.210596

            REGN        CMS
count  41.000000  41.000000
mean    0.002558  -0.002767
std     0.041395   0.051880
min    -0.110813  -0.131082
25%    -0.021965  -0.035108
50%     0.001222   0.001375
75%     0.021954   0.026691
max     0.109189   0.129441

[8 rows x 30 columns]
```

Date	JPM	WFC	BAC	C	GS	USB	MS	KEY	PNC	COF	...	CSCO	VTR	EQIX	DU
2020-03-03 00:00:00-	-0.038247	-0.041799	-0.056738	-0.038303	-0.029259	-0.043716	-0.045783	-0.045818	-0.055413	-0.056792	...	-0.027831	-0.004019	-0.006029	-0.01102

05:00															
2020-03-04 00:00:00-05:00	0.024409	0.021239	0.022801	0.035340	0.025768	0.012075	0.018742	0.029181	0.030682	0.033118	...	0.033160	0.030219	0.048127	0.06125
2020-03-05 00:00:00-05:00	-0.050306	-0.062287	-0.052021	-0.059614	-0.048840	-0.063711	-0.060362	-0.043472	-0.069683	-0.048745	...	-0.044968	-0.037146	-0.045676	-0.01506
2020-03-06 00:00:00-05:00	-0.053064	-0.047647	-0.040776	-0.035429	-0.030336	-0.030850	-0.017767	-0.071992	-0.055011	-0.033342	...	0.002776	-0.042803	-0.013057	-0.01084
2020-03-09 00:00:00-04:00	-0.145552	-0.132723	-0.159024	-0.176399	-0.109720	-0.155959	-0.109512	-0.201194	-0.145622	-0.118832	...	-0.044314	-0.099511	-0.058542	-0.04617
2020-03-10 00:00:00-04:00	0.074826	0.077007	0.073815	0.078769	0.064643	0.061920	0.051960	0.094861	0.089910	0.069768	...	0.054344	-0.022058	0.061184	0.00495
2020-03-11 00:00:00-04:00	-0.048214	-0.081636	-0.040628	-0.090124	-0.069981	-0.064953	-0.068903	-0.087487	-0.058117	-0.092608	...	-0.078609	-0.146292	-0.018990	-0.04420
2020-03-12 00:00:00-04:00	-0.086027	-0.172779	-0.100130	-0.160471	-0.131697	-0.135862	-0.163461	-0.119519	-0.073459	-0.115493	...	-0.109718	-0.327085	-0.057872	-0.12219
2020-03-13 00:00:00-04:00	0.165620	0.127216	0.163786	0.165381	0.161951	0.160143	0.180403	0.157955	0.121717	0.135180	...	0.125517	0.197010	0.094420	0.06366
2020-03-16 00:00:00-04:00	-0.162106	-0.153288	-0.167205	-0.214414	-0.135880	-0.130225	-0.169603	-0.242377	-0.173166	-0.272783	...	-0.110273	-0.336772	-0.135417	-0.11333
2020-03-17 00:00:00-04:00	0.059319	0.111643	0.072639	-0.023085	0.025597	0.034699	0.065427	0.024717	0.064872	-0.027980	...	0.051738	-0.124211	0.058624	0.11601
2020-03-18 00:00:00-04:00	-0.111232	-0.052306	-0.055661	-0.099718	-0.125041	-0.021584	-0.094275	-0.227057	-0.092367	-0.196517	...	0.044623	-0.214028	-0.005299	-0.08136

2020-03-19 00:00:00-04:00	0.016668	0.006027	0.019529	0.084446	0.065444	0.006344	0.005513	0.136132	0.054362	0.043293	...	0.015770	0.163285	-0.039465	-0.05483
2020-03-20 00:00:00-04:00	-0.021328	-0.065364	-0.074907	-0.040675	-0.077009	-0.019460	-0.041267	-0.017261	-0.032217	-0.027393	...	-0.057580	0.078887	-0.082355	-0.09278
2020-03-23 00:00:00-04:00	-0.055019	-0.048319	-0.084288	-0.072735	-0.025168	-0.118513	-0.064741	-0.071016	-0.104132	-0.048712	...	-0.028492	0.061876	-0.034915	-0.06414
2020-03-24 00:00:00-04:00	0.112384	0.135707	0.151144	0.138816	0.129299	0.075539	0.177757	0.132216	0.111813	0.171887	...	0.109399	0.110996	0.109769	0.10847
2020-03-25 00:00:00-04:00	0.036638	0.001727	0.003323	0.029086	0.009912	0.026255	0.022914	0.005104	0.020524	0.027701	...	-0.024389	0.099898	0.014602	0.04391
2020-03-26 00:00:00-04:00	0.067342	0.064495	0.073973	0.094745	0.066458	0.121834	0.049364	0.154442	0.111620	0.100438	...	0.074411	0.011553	0.058992	0.08317
2020-03-27 00:00:00-04:00	-0.073904	-0.020269	-0.050552	-0.049442	-0.045977	-0.011397	-0.048776	-0.022949	-0.027483	-0.045426	...	-0.044340	0.056172	0.011321	-0.01239
2020-03-30 00:00:00-04:00	0.025675	-0.011960	0.020166	0.006372	0.008051	0.014603	0.024112	-0.037294	0.008354	-0.026199	...	0.037912	-0.054085	0.081715	0.03996
2020-03-31 00:00:00-04:00	-0.037818	-0.041630	-0.037444	-0.045484	-0.032019	-0.040392	-0.024406	-0.039703	-0.053396	-0.066186	...	-0.025369	-0.043696	-0.031985	-0.03140
2020-04-01 00:00:00-04:00	-0.065049	-0.077114	-0.071250	-0.089605	-0.062045	-0.075963	-0.072571	-0.092902	-0.091504	-0.120862	...	-0.025246	-0.155084	-0.047729	-0.04178
2020-04-02 00:00:00-04:00	0.036659	0.024169	0.039668	0.018524	0.031437	0.022911	0.069625	0.003169	0.037965	-0.016929	...	0.047070	0.002611	0.050256	0.02319

2020-04-03 00:00:00-04:00	-0.030003	-0.037048	-0.026602	-0.045368	-0.020212	-0.046360	-0.001476	-0.044212	-0.063004	-0.038520	...	-0.018768	-0.021525	-0.002334	-0.04337
2020-04-06 00:00:00-04:00	0.062379	0.087551	0.065692	0.092420	0.074093	0.097086	0.089249	0.092600	0.112251	0.156277	...	0.058906	0.149063	0.043126	0.06653
2020-04-07 00:00:00-04:00	0.013104	0.004878	0.034463	0.003157	0.048059	0.002325	-0.001893	0.049029	-0.004100	0.068237	...	-0.019253	0.035329	-0.044840	-0.01413
2020-04-08 00:00:00-04:00	0.039586	0.051154	0.057484	0.070430	0.063815	0.034240	0.063709	0.080852	0.047123	0.069535	...	0.026707	0.107389	0.063132	0.06511
2020-04-09 00:00:00-04:00	0.085915	0.092062	0.058390	0.068752	0.040424	0.065152	0.042517	0.058286	0.057795	0.058049	...	-0.013022	0.114276	0.033573	0.05473
2020-04-13 00:00:00-04:00	-0.045492	-0.054787	-0.038545	-0.015518	-0.027957	-0.033675	-0.037198	-0.038189	-0.043228	-0.061936	...	0.000486	-0.037376	-0.044274	-0.03321
2020-04-14 00:00:00-04:00	-0.027778	-0.040583	-0.007975	-0.027363	-0.005316	-0.027278	0.007049	-0.037008	-0.014868	-0.020745	...	0.037147	0.018863	0.047108	0.03442
2020-04-15 00:00:00-04:00	-0.050577	-0.059383	-0.067098	-0.058013	0.001626	-0.071490	-0.037316	-0.077428	-0.061776	-0.050780	...	-0.029896	-0.083565	-0.019584	-0.03236
2020-04-16 00:00:00-04:00	-0.038855	-0.056042	-0.035317	-0.056143	-0.008325	-0.047621	-0.001042	-0.056873	-0.020660	-0.050493	...	-0.000482	-0.027235	0.010616	0.00057
2020-04-17 00:00:00-04:00	0.086076	0.053930	0.083270	0.114817	0.035785	0.098224	0.018851	0.104231	0.114267	0.088360	...	0.023340	0.052151	0.017726	0.02825
2020-04-20 00:00:00-04:00	-0.037139	-0.024974	-0.034079	-0.032196	-0.016983	-0.023374	-0.018851	0.011040	0.010292	-0.014009	...	0.001411	-0.088145	-0.035444	-0.03904
2020-04-21															

00:00:00-04:00	-0.029434	-0.030817	-0.038972	-0.057039	-0.037214	-0.045395	-0.037450	-0.057428	-0.035231	-0.051616	...	-0.047909	-0.025452	0.002160	-0.01406
2020-04-22 00:00:00-04:00	0.003251	-0.001491	0.007366	0.015989	0.012408	0.015763	0.023008	-0.008759	-0.001011	0.007012	...	0.029403	-0.004306	0.018556	0.01981
2020-04-23 00:00:00-04:00	0.000560	-0.010126	0.003206	0.005195	-0.005299	0.003602	-0.009033	0.038356	0.005345	0.020936	...	-0.003358	0.011085	0.001022	-0.02356
2020-04-24 00:00:00-04:00	0.014659	0.014593	0.014075	0.014961	0.011078	0.018703	0.013256	0.025082	0.012990	0.065121	...	0.021393	-0.014688	-0.000569	0.00667
2020-04-27 00:00:00-04:00	0.042201	0.053872	0.056532	0.077219	0.036283	0.053820	0.034683	0.063104	0.046173	0.052726	...	0.012620	0.027409	0.024368	0.01229
2020-04-28 00:00:00-04:00	0.007056	0.016062	0.017737	0.013864	0.018729	0.021233	0.020645	0.018772	0.004729	0.074217	...	-0.013326	0.023596	-0.026326	0.01009
2020-04-29 00:00:00-04:00	0.026613	0.038394	0.036577	0.062604	0.016020	0.048205	0.025073	0.034887	0.046095	0.089948	...	0.019808	0.113353	-0.006514	-0.01147

41 rows × 30 columns

Step 11: Rerun the Algorithms (Both UCB and Epsilon-Greedy)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ==================================================================
# ALGORITHM DEFINITIONS (FROM STEP 6 & 8)
# We include the functions here for a complete, runnable script.
# ==================================================================

def ucb_algorithm(returns_df, c=2):
    """Implements the Upper-Confidence-Bound (UCB) algorithm."""
    n_rounds, n_arms = returns_df.shape
    counts = np.zeros(n_arms)
    values = np.zeros(n_arms)
    history = []
    for t in range(n_rounds):
        max_upper_bound = -np.inf
        selected_arm = 0
        for k in range(n_arms):
            if counts[k] > 0:
                average_reward = values[k] / counts[k]
                exploration_bonus = c * np.sqrt(np.log(t + 1) / counts[k])
                upper_bound = average_reward + exploration_bonus
            else:
                upper_bound = np.inf
            if upper_bound > max_upper_bound:
                max_upper_bound = upper_bound
                selected_arm = k
        history.append(selected_arm)
        reward = returns_df.iloc[t, selected_arm]
        counts[selected_arm] += 1
        values[selected_arm] += reward
    return history

def epsilon_greedy_algorithm(returns_df, epsilon=0.1):
    """Implements the Epsilon-Greedy algorithm."""
    n_rounds, n_arms = returns_df.shape
    q_values = np.zeros(n_arms)
    arm_counts = np.zeros(n_arms)
    history = []
    for t in range(n_rounds):
        if np.random.rand() < epsilon:
            chosen_arm = np.random.randint(n_arms)
        else:
            chosen_arm = np.argmax(q_values)
```

```
            history.append(chosen_arm)
            reward = returns_df.iloc[t, chosen_arm]
            arm_counts[chosen_arm] += 1
            q_values[chosen_arm] += (1 / arm_counts[chosen_arm]) * (reward - q_values[chosen_arm])
        return history

# ================================================================
# STEP 11: ANALYSIS
# ================================================================

# --- 1. Load both datasets ---
print("Loading datasets for 2008 and 2020...")
try:
    returns_2008_df = pd.read_csv('log_returns_df.csv', index_col='Date', parse_dates=True)
    returns_2020_df = pd.read_csv('log_returns_2020.csv', index_col='Date', parse_dates=True)
    print("Datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: Could not find a required data file. {e}")
    print("Please ensure 'log_returns_df.csv' and 'log_returns_2020.csv' are present.")
    exit()

# --- 2. Run algorithms on both datasets ---
print("Running MAB algorithms on both datasets...")
# 2008 Results
ucb_hist_2008 = ucb_algorithm(returns_2008_df, c=2)
eg_hist_2008 = epsilon_greedy_algorithm(returns_2008_df, epsilon=0.1)

# 2020 Results
ucb_hist_2020 = ucb_algorithm(returns_2020_df, c=2)
eg_hist_2020 = epsilon_greedy_algorithm(returns_2020_df, epsilon=0.1)
print("Algorithm runs complete.")

# --- 3. Calculate cumulative rewards for all scenarios ---
def calculate_cumulative_rewards(returns_df, history):
    rewards = [returns_df.iloc[t, history[t]] for t in range(len(history))]
    return np.cumsum(rewards)

ucb_cum_2008 = calculate_cumulative_rewards(returns_2008_df, ucb_hist_2008)
eg_cum_2008 = calculate_cumulative_rewards(returns_2008_df, eg_hist_2008)
ucb_cum_2020 = calculate_cumulative_rewards(returns_2020_df, ucb_hist_2020)
eg_cum_2020 = calculate_cumulative_rewards(returns_2020_df, eg_hist_2020)

# --- 4. Create Final Comparison Dashboard ---
print("Generating final comparison dashboard...")
fig, axes = plt.subplots(2, 2, figsize=(22, 18))
fig.suptitle('MAB Algorithm Performance: 2008 Financial Crisis vs. 2020 COVID-19 Crash', fontsize=22)

# Plot 1: 2008 Performance
axes[0, 0].plot(returns_2008_df.index[:len(ucb_cum_2008)], ucb_cum_2008, label='UCB')
axes[0, 0].plot(returns_2008_df.index[:len(eg_cum_2008)], eg_cum_2008, label='Epsilon-Greedy')
```

```
axes[0, 0].plot(returns_2008_df.index[:len(eg_cum_2008)], eg_cum_2008, label='Epsilon-Greedy')
axes[0, 0].set_title('Performance during 2008 Financial Crisis', fontsize=16)
axes[0, 0].set_ylabel('Cumulative Log Return', fontsize=12)
axes[0, 0].legend()
axes[0, 0].grid(True)

# Plot 2: 2020 Performance
axes[0, 1].plot(returns_2020_df.index, ucb_cum_2020, label='UCB')
axes[0, 1].plot(returns_2020_df.index, eg_cum_2020, label='Epsilon-Greedy')
axes[0, 1].set_title('Performance during 2020 COVID-19 Crash', fontsize=16)
```